

SmartBIS Nginx 사설 인증서 생성 및 HTTPS 테스트 기록 v1.0

이 문서는 SmartBIS 신규 프로젝트에서 Nginx 사설 CA를 생성하고, Spring Boot API까지 HTTPS로 연결한 과정과 발생한 문제 및 해결 방법을 기록한다.

1. 최종 검증 결과

다음 통신 경로가 정상 동작했다.

```
Windows client
  ↓ https://localhost:8443
Nginx container :443
  ↓ http://egov-backend:8080/api
Spring Boot embedded WAS
  ↓ jdbc:postgresql://postgres-db:5432/smartbis_db
PostgreSQL container
```

최종 검증 명령:

```
curl --cacert server/nginx/certs/rootCA.crt --ssl-no-revoke
https://localhost:8443/api/actuator/health
```

정상 응답:

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP"
    }
  }
}
```

이는 단순히 TLS 오류를 무시한 테스트가 아니라 **rootCA.crt**를 사용해 인증서 체인을 검증한 결과다.

2. 인증서 파일의 역할

rootCA.key	사설 Root CA 개인키. 절대 공유하거나 Git에 저장하지 않음
rootCA.crt	Android와 테스트 클라이언트가 신뢰할 CA 인증서
server.key	Nginx 서버 개인키. 외부 공개 금지
server.csr	서버 인증서 서명 요청 파일
server.crt	Nginx가 사용하는 서버 인증서

```
server.ext  서버 인증서의 SAN과 사용 용도 설정
rootCA.srl  CA 서명 시 생성되는 일련번호 파일
```

`.gitignore`에는 다음을 포함한다.

```
server/nginx/certs/*.key
server/nginx/certs/*.csr
server/nginx/certs/*.srl
server/nginx/certs/rootCA.*
server/nginx/certs/server.*
```

운영에서 인증서 개인키를 분실하면 새 서버 인증서를 발급해야 하며, Root CA 개인키가 유출되면 CA 자체를 폐기하고 새로 만들어야 한다.

3. 인증서 생성 위치

```
cd /d/PROJECT/SmartBIS_jangsung/server/nginx/certs
```

모든 인증서는 Nginx Compose 볼륨으로 마운트되는 이 디렉터리에 생성한다.

4. Root CA 생성

```
openssl genrsa -out rootCA.key 4096

openssl req -x509 -new -nodes \
  -key rootCA.key \
  -sha256 \
  -days 3650 \
  -out rootCA.crt \
  -subj "//CN=SmartBIS Local Root CA"
```

Git Bash에서 줄바꿈이 제대로 전달되지 않으면 한 줄로 실행한다.

```
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -out rootCA.crt -
subj "//CN=SmartBIS Local Root CA"
```

`rootCA.crt`는 Android 앱에 배포할 수 있지만 `rootCA.key`는 절대 배포하지 않는다.

5. 서버 인증서 SAN 설정

서버 인증서는 접속 주소와 SAN이 일치해야 한다. 개발 환경에서 사용할 주소는 다음 세 가지다.

```
https://localhost:8443
https://127.0.0.1:8443
https://10.0.2.2:8443
```

따라서 `server.ext`를 다음처럼 생성한다.

```
cat > server.ext <<'EOF'
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=@alt_names

[alt_names]
DNS.1=localhost
IP.1=127.0.0.1
IP.2=10.0.2.2
EOF
```

CN만 `localhost`로 설정하는 것으로는 IP 주소 접속 검증을 충분히 처리할 수 없다. 실제 접속 주소는 SAN에 기록해야 한다.

6. 서버 인증서 발급

```
openssl genrsa -out server.key 2048

openssl req -new \
  -key server.key \
  -out server.csr \
  -subj "//CN=localhost"

openssl x509 -req \
  -in server.csr \
  -CA rootCA.crt \
  -CAkey rootCA.key \
  -CAcreateserial \
  -out server.crt \
  -days 825 \
  -sha256 \
  -extfile server.ext
```

생성 결과 확인:

```
ls -lh
openssl x509 -in server.crt -noout -subject -issuer -dates
```

정상 예시:

```
subject=CN=localhost
issuer=CN=SmartBIS Local Root CA
notBefore=...
notAfter=...
```

7. SAN 확인

```
openssl x509 -in server.crt -noout -text | grep -A2 "Subject Alternative Name"
```

정상 결과:

```
X509v3 Subject Alternative Name:
  DNS:localhost, IP Address:127.0.0.1, IP Address:10.0.2.2
```

SAN이 누락되었으면 기존 `server.crt`를 재사용하지 말고 `server.ext`를 수정한 뒤 서버 인증서를 다시 발급한다.

8. Nginx 인증서 마운트와 설정

`compose.yaml`:

```
nginx:
  image: docker.io/library/nginx:alpine
  ports:
    - "8443:443"
  volumes:
    - ./server/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./server/nginx/certs:/etc/nginx/certs:ro
  depends_on:
    - egov-backend
```

`nginx.conf`의 인증서 경로:

```
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;
```

API 전달 경로:

```
location /api/ {
    proxy_pass http://egov-backend:8080/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
}
```

9. Nginx 설정 자체 검증

```
podman exec smartbis_jangsung-nginx-1 nginx -t
```

정상 결과:

```
syntax is ok
test is successful
```

이 검사는 Nginx 설정 문법과 인증서 파일을 읽을 수 있는지 확인한다. 이 검사가 성공해도 Windows 호스트 포트 전달 문제까지 해결되었다는 뜻은 아니다.

10. 컨테이너 내부 HTTPS 검증

Nginx 컨테이너 IP를 확인한다.

```
podman inspect smartbis_jangsung-nginx-1 --format '{{range
.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
```

확인한 IP가 10.89.1.4인 경우:

```
podman run --rm --network smartbis_jangsung_default
docker.io/curlimages/curl:8.10.1 -vk https://10.89.1.4/api/actuator/health
```

정상 결과:

```
TLSv1.3 connection
HTTP/1.1 200
{"status":"UP", ...}
```

이 테스트가 성공하면 Nginx, 인증서, Spring Boot, PostgreSQL은 정상이고, 남은 문제는 호스트에서 컨테이너로의 포트 전달이다.

11. Windows 포트 전달 문제와 진단

초기에는 다음 상황이 발생했다.

```
podman port: 443/tcp -> 0.0.0.0:8443
Windows localhost:8443: connection refused
```

다음 명령으로 확인한다.

```
netstat -ano | grep 8443
podman port smartbis_jangsung-nginx-1
podman machine inspect
```

`podman port`에 매핑이 표시되어도 Windows에 실제 LISTENING 소켓이 없을 수 있다. 이 경우 애플리케이션이나 Nginx를 수정하기 전에 Podman machine 네트워크 설정을 확인한다.

12. Rootless 및 User Mode Networking 전환

Windows/WSL Podman 환경에서 Rootless와 User Mode Networking을 사용하려면 다음을 실행한다.

```
podman machine stop
podman machine set --user-mode-networking=true podman-machine-default
podman machine set --rootful=false podman-machine-default
podman machine start
```

확인:

```
podman machine inspect | grep -E 'Rootful|UserModeNetworking'
```

정상 결과:

```
"UserModeNetworking": true
"Rootful": false
```

이 설정 변경 후에는 기존 rootful 컨테이너와 별도의 Rootless 저장소가 사용될 수 있다. PostgreSQL 볼륨을 삭제하지 않고 새 Compose 프로젝트를 다시 올린다.

```
cd /d/PROJECT/SmartBIS_jangsung
podman compose up -d
podman compose ps
```

정상적으로 Windows에 리스너가 생성되는지 확인한다.

```
netstat -ano | grep 8443
```

13. Windows curl의 IP 접속 문제

다음 방식은 인증서가 **CN=localhost**인 경우 피한다.

```
curl -vk https://127.0.0.1:8443/api/actuator/health
```

Windows Schannel이 IP 주소 접속에서 SNI를 사용하지 못해 다음 오류가 발생할 수 있다.

```
schannel: using IP address, SNI is not supported by OS
schannel: SSL/TLS connection failed
```

개발 PC에서는 SAN의 DNS 이름과 일치하는 **localhost**를 사용한다.

```
curl -vk https://localhost:8443/api/actuator/health
```

14. -k 테스트와 실제 CA 검증의 차이

-k는 인증서 검증을 전부 끄는 옵션이다.

```
curl -k https://localhost:8443/api/actuator/health
```

연결성 확인에는 유용하지만 인증서 신뢰성 검증은 하지 않는다.

사실 CA를 실제로 검증하려면 **rootCA.crt**를 지정한다.

```
curl --cacert server/nginx/certs/rootCA.crt
https://localhost:8443/api/actuator/health
```

15. Windows Schannel 폐기 상태 오류

다음 오류가 발생할 수 있다.

```
schannel: the revocation status is unknown
```

로컬 사설 CA에는 공개 인증서처럼 연결된 CRL/OCSP 폐기 확인 서버가 없기 때문에 Windows가 폐기 상태를 확인하지 못하는 것이다. 인증서 체인 자체의 오류와 구분해야 한다.

개발 테스트에서는 인증서 체인 검증은 유지하고 폐기 상태 확인만 생략한다.

```
curl --cacert server/nginx/certs/rootCA.crt --ssl-no-revoke  
https://localhost:8443/api/actuator/health
```

정상 응답이 나오면 다음을 모두 검증한 것이다.

- **rootCA.crt**가 서버 인증서를 발급함
- **server.crt**의 SAN이 **localhost**와 일치함
- Nginx가 올바른 인증서를 제공함
- Windows가 CA 체인을 신뢰함
- Nginx가 Spring Boot로 요청을 전달함

16. Android 적용 시 주의사항

Android Emulator의 서버 주소:

```
https://10.0.2.2:8443
```

Android 앱에는 **rootCA.crt**만 포함한다. 다음 파일은 포함하지 않는다.

```
rootCA.key  
server.key
```

Android 앱의 인증서 검증은 OkHttp/Network Security Config 정책으로 구현하고, 개발 편의를 위해 모든 인증서를 허용하는 TrustManager를 사용하지 않는다.

17. 위험한 명령

다음 명령은 현재 PostgreSQL 데이터를 삭제할 수 있으므로 데이터 보존 여부를 확인하기 전에는 실행하지 않는다.

```
podman compose down -v  
podman volume rm ...  
podman system reset
```

```
podman machine reset  
podman machine rm
```

단순히 컨테이너를 중지하거나 재생성할 때는 다음을 우선 사용한다.

```
podman compose down  
podman compose up -d
```